EURS-Net V2: Team Pre-Flight Engineering Report

A Comprehensive, Plain-English Guide to Our Pre-GPU Work & RTX 5090 Launch Plan

Challenge: Embedded Underwater Restoration & Segmentation (EURS) — **Conference:** MaCVi @ WACV 2027

Team Division: Tanmay (Modeling & GPU) Shubham (Data & Deployment) Arin (Tooling & Packaging)

Deployment Target: NVIDIA Jetson AGX Orin (Track 2) **Training Cluster:** NVIDIA RTX 5090 (Blackwell 32GB)

Date: October 2026 — **Status:** Pre-Flight Invariants 100% Verified (Zero Known Software Defects)

---

1 Introduction & The Challenge in Plain English

Autonomous underwater robots face a double challenge: ocean water heavily absorbs red light (causing intense green/blue haze and blurry contrast), while onboard computers must simultaneously detect marine life (corals, fish, sharks).

Normally, robots run two bulky AI models: one to fix colors, and another to segment objects. However, the **WACV 2027 Track 2 rules enforce strict physical limits** so code can run on a compact, battery-powered **NVIDIA Jetson AGX Orin (30W)**:

Parameter Budget: Maximum 1,000,000 parameters. *Our EURS-Net V2 uses only **150,438 parameters** (∼85% below cap).*

Compute Budget: Maximum 12.0 GFLOPs. *Our EURS-Net V2 requires only **4.47 GFLOPs** (∼63% below cap).*

Composite Competition Score: $S = 0.60 \times Q_{\text{seg}} + 0.30 \times Q_{\text{rest}} + 0.10 \times Q_{\text{NR}}$ (60% Segmentation, 30% Restoration, 10% Quality).

Before spending valuable compute hours on the new **RTX 5090** cluster, our team completely decoupled the project into three distinct modular domains, testing and verifying every algorithm locally on CPU first.

2 Work Accomplished So Far (Pre-GPU Development)

2.1 1. Tanmay Das: Model Architecture, Loss Math & Pre-Flight Testing

2.1.1 A. The EURS-Net V2 Architecture (How it Works)

Tanmay built an ultra-efficient multi-task neural network that processes a single 1024×1024 degraded underwater image and generates two outputs at once: a clean restored RGB image and a 5-class segmentation mask.

Fast White-Balance Pre-Net: Instead of forcing the deep network to struggle with water discoloration, 6 learnable parameters immediately correct red/green/blue gains at the very input before deep feature extraction.

Shared MobileNetV2 Backbone with Coordinate Attention (CA): Standard 2D attention is too slow for 1024px images. Coordinate Attention pools along horizontal and vertical axes separately, capturing exact object positions (like small fish) with minimal compute.

Dual Decoders: A lightweight *Restoration Decoder* predicts a residual difference map (ΔI), producing clean RGB via $\text{Clamp}(\text{Input} + \Delta I, 0, 1)$, while a *Feature Pyramid Segmentation Decoder* with a Small-Object Neck predicts 5 semantic classes.

2.1.2 B. The Multi-Task Loss Objective in Detail

In multi-task learning, if you just add two losses together naively, one task bullies the other (e.g., the model learns to restore colors beautifully but completely forgets how to detect fish). Tanmay engineered a mathematically balanced objective:

$$\mathcal{L}_{\text{total}} = 1.0 \cdot \mathcal{L}_{\text{seg}} + 0.5 \cdot \mathcal{L}_{\text{rest}}$$

Why the 1.0 to 0.5 ratio? This directly mirrors the competition's evaluation formula, which values segmentation (60%) twice as much as restoration (30%).

1. **The Segmentation Objective ($\mathcal{L}_{\text{seg}} = \mathcal{L}_{\text{Focal}} + \mathcal{L}_{\text{Dice}}$):**

   Class-Weighted Focal Loss: In underwater imagery, $> 90\%$ of pixels are empty background water. A standard cross-entropy loss would let the model cheat by guessing "background" everywhere. Focal Loss uses a modulating factor $(1 - p_t)^{\gamma}$ to discount easy background pixels and focus gradient updates heavily on hard, rare objects (like small fish and sharks).

   Soft Dice Loss: Instead of looking at individual pixels in isolation, Dice Loss directly measures the spatial overlap (Intersection over Union) between the predicted creature mask and the ground-truth mask. This forces the model to capture realistic, coherent object shapes.

   Mean-1.0 Square-Root Smoothed Weights (α_c): Rare classes get boosted loss weights proportional to $1/\sqrt{\text{Frequency}_c}$. Crucially, Tanmay normalized these weights so their mean is exactly 1.0, which prevents the loss from exploding in scale.

2. **The Restoration Objective ($\mathcal{L}_{\text{rest}} = \mathcal{L}_{\text{Charbonnier}} + 2.0 \cdot \mathcal{L}_{\text{MS-SSIM}} + 0.1 \cdot \mathcal{L}_{\text{Laplacian}}$):**

Charbonnier Loss: A smooth, differentiable variant of L1 loss ($\sqrt{(I_{\text{pred}} - I_{\text{clean}})^2 + \epsilon^2}$). It accurately measures pixel-by-pixel color fidelity without causing gradient instability near zero error.
Multi-Scale Structural Similarity (MS-SSIM): Measures contrast, luminance, and texture across multiple downsampled resolution scales. This ensures that the restored water and reefs look natural and structurally sound to human perception.
Laplacian Edge Loss: Uses a 3×3 Laplacian second-derivative filter (∇^2) to isolate edges. It penalizes blurry object outlines, directly optimizing the model to maximize the official competition's Sharpness Ratio metric.

2.1.3 C. The 20-Suite Pre-Flight Verification Engine (Explained Simply)

Before touching the RTX 5090, Tanmay created 20 automated unit test suites. Think of these as a multi-stage pre-flight checklist:

1. **The Budget Police (`test_architecture.py`):** Verifies that total parameters (150K $\leq$ 1M) and FLOPs (4.47G $\leq$ 12.0G) strictly comply with competition hardware caps before any code is trained.
2. **The Data Pairing Verifier (`test_dataset_integrity.py`):** Crawls the dataset to confirm that every degraded image perfectly matches its corresponding clean photo and mask label with 0% file mismatch.
3. **The Lockstep Transform Check (`test_transform_alignment.py`):** Creates a synthetic 4-quadrant colored image with matching mask. When the pipeline randomly flips or crops the image, this test proves the mask flips and crops at the *exact same pixel coordinates*.
4. **The Small-Object Survival Test (`test_small_object_preservation.py`):** Injects synthetic 1-pixel and 2-pixel fish into images to mathematically guarantee that random cropping never slices away tiny marine animals.
5. **The Brain Isolation Test (`test_losses_and_gradients.py`):** Proves that backpropagating segmentation loss updates only the segmentation decoder, leaving the restoration decoder completely untouched (clean modularity).
6. **The Rare-Pixel Math Guard (`test_loss_stability_full.py`):** Proves that even if an image contains only a single 1-pixel shark against 1,000,000 background pixels, the focal loss calculates stable, finite gradients without turning into `NaN` (infinity).
7. **The Referee Math Audit (`test_metric_fixtures.py`):** Feeds known mathematical test fixtures into the evaluation code to ensure mIoU, F1, PSNR, SSIM, and composite score calculations are 100% mathematically accurate.
8. **The 10-Sample Memorization Test (`test_overfit.py`):** Trains the network on just 4 real images repeatedly for 50 steps. The loss plummeted by $> 50\%$, proving the forward pass, backward pass, and loss math can learn real patterns.
9. **The Pixel Clamping Check (`test_output_clamp.py`):** Guarantees that restored colors are strictly bounded between 0.0 and 1.0 (no impossible negative or radioactive colors) without breaking gradient flow.
10. **The Hardware Parity Test (`test_device_parity.py`):** Runs the model on CPU and GPU side-by-side; confirms outputs match within < 0.0001 tolerance.
11. **The Learning Rate Dynamics Test (`test_lr_scheduler.py`):** Verifies that learning rate smoothly warms up linearly for 5 epochs and decays via cosine curve without dropping below minimum thresholds.
12. **The Minimal Training Loop (`train_debug.py`):** A 1-epoch dry-run on CPU verifying batch collation, loss steps, and metric evaluations end-to-end.
13. **The Jetson Robot Contract (`test_onnx_contract.py`):** Exports the model to static ONNX format (Opset 18) and confirms that ONNX Runtime matches PyTorch outputs down to $< 10^{-4}$ precision.
14. **The Checkpoint Roundtrip Test (`test_checkpoint_roundtrip.py`):** Saves model weights to disk, reloads them into a new instance, and confirms numerical predictions are 100% identical.
15. **The Training Resume Determinism Test (`test_resume_determinism.py`):** Simulates training interruption: 5 steps continuous vs. 2 steps $\rightarrow$ save $\rightarrow$ reload $\rightarrow$ 3 steps. Confirms weights and optimizer momentum buffers match bitwise.
16. **The Sampler Distribution Test (`test_samplers.py`):** Confirms the custom sampler balances rare and common video scenes evenly across batches.
17. **The Submission Format Guard (`test_submission_track1_format.py`):** Generates synthetic predictions, packages them into a ZIP, and verifies single-channel PNG formats and root folder layout.
18. **The Preprocessing Cache Test (`test_cache`$_{e}$*quivalence.py*) : $Verifiesthatpre-cachingresizedimagespreserves100\%bi$ *casescenarios* : $pitch-blackinputs, pure-whiteinputs, oddbatchsizes, andmulti-workerDataLoaderconcurrency.$**
19. **The RTX 5090 Blackwell Smoke Test (`test_cuda`$_{d}$*eterminism.py*) : $TestsGPUvisibility, Blackwell$`sm_120`$architectureco$**

2.2 2. Shubham: Data Engineering, Scientific Splits, Evaluation & Deployment

Scientific 3-Tier Validation Protocol (`data/splits.json`): Discovered that Class 3 (sharks/rays) only exists in 2 scenes, and Class 4 in 3 scenes. Shuffling randomly would cause catastrophic data leakage or leave rare classes out of training. Shubham engineered:

- *Tier 1 (Canonical 16/4 Split):* Hand-balanced daily split ensuring rare classes exist in both Train and Val with 0% leakage.
- *Tier 2 (Rare-Class Swap):* A 2-run swap protocol for making major architectural decisions.
- *Tier 3 (4-Fold Out-of-Fold Aggregation):* 4 folds of 15 train / 5 val scenes, aggregating complete OOF

predictions across all 20 scenes for release benchmarking.

Synchronized Data Pipeline Specification (`data/`): Designed paired triplet loading (`degraded`, `clean`, `mask`) with synchronized spatial transforms (horizontal flips, spatial crops) while strictly banning uncalibrated color jitter to safeguard white-balance learning.
Authoritative Evaluation Engine (`metrics/`, `evaluate.py`): Implemented the official scoring formulas: confusion matrix mIoU, Macro F_1, Normalized PSNR (15–35 dB window), SSIM, and Laplacian sharpness ratio. Added dual-lens reporting (**Global Score** + **Scene-Macro Diagnostic Scorecard**) so easy scenes cannot mask failures on rare creatures.
Static ONNX Deployment Contract (`deploy/`): Authored the static ONNX exporter for Jetson Orin, reserving modeling freedom to test alternative resolutions (384×768 vs. 1024×1024) to find the highest score under the 12 GFLOP ceiling.

2.3 3. Arin: Tooling, Exploratory Data Analysis, Visualization & Packaging

Exploratory Data Analysis (`tools/eda_stats.py`): Counted pixels across all 5 classes, calculated exact imbalance ratios, and cataloged camera resolutions (4000×3000 to 9710×7283).
Qualitative Diagnostic Visualizer (`tools/visualizer.py`): Built side-by-side visual plotting tools rendering degraded inputs, clean references, masks, and model prediction overlays for error diagnosis and research paper figures.
Track 1 Submission Packager (`tools/make_submission_track1.py`): Built the automated packager that upscales model predictions back to original camera dimensions (using bicubic for RGB and nearest-neighbor for masks), ensures single-channel `uint8` PNGs, and packages `submission_track1.zip` without format errors.
Experiment Telemetry (`experiments/ablations.md`): Maintains the benchmark log recording experiment IDs, parameter counts, GFLOPs, PSNR, SSIM, and mIoU.

3 Modular Work Division Matrix

Functional Domain	Tanmay (Modeling & Training)	Shubham (Data & Deployment)	Arin (Tooling & Packaging)
Neural Architecture	EURS-Net V2 (FWB + CA + Decoders)	Budget audits & FLOP compliance	Records parameter logs
Loss Formulations	Focal, Soft Dice, Charbonnier, MS-SSIM	Evaluates loss weighting impact	Tracks loss convergence curves
Dataset Management	Consumes DataLoader batches	3-Tier Split Manifests & Pipeline	EDA pixel counts & class ratios
Testing & Auditing	20 Automated verification suites	Overfit test harness & parity checks	Visual prediction comparisons
Evaluation Metrics	Calls eval hooks during training	Computes Global & Scene-Macro scores	Logs leaderboard benchmarks
Final Deployment	Produces trained weights (`.pt`)	Exports static Track 2 ONNX	Packages Track 1 submission ZIP

4 Immediate Action Steps Once Hands-On With RTX 5090

When accessing the RTX 5090 environment, our team will execute this sequenced 8-step roadmap:

1. **Step 1: GPU Capability Smoke Test (∼10s)**
   Verify Python 3.10+, PyTorch with CUDA 12.8+, and Blackwell support (`sm_120`). Run GPU smoke test:
   `python tests/test_cuda_determinism.py` *(Verifies 32GB VRAM, native BF16, finite gradients).*
2. **Step 2: Heavy Edge-Case Stress Audit (∼15s)**
   Execute the 14-point edge-case suite to ensure zero runtime panics under extreme batches or empty masks:
   `python -m unittest tests/test_heavy_edge_cases.py`
3. **Step 3: Physical Dataset Extraction & Directory Harmonization (∼2m)**
   Extract `eurs_train.zip` to storage directory. Ensure discovery via `find_dataset_subdirs()` or symlinks:
   `cd data && ln -s gt_images clean && ln -s gt_masks masks`
4. **Step 4: End-to-End Phase 2A Pre-Flight Gatekeeper (∼1m)**
   Run the complete gatekeeper from repository root:
   `python tests/test_phase2_preflight.py` *(100% PASS across all 8 work packages).*
5. **Step 5: Training Engine Micro Dry-Run (∼20s)**
   Validate the production training engine using the dry-run guardrail (executes 2 batches and 1 validation batch):
   `python train.py --config configs/baseline_v1.yaml --dry_run`
6. **Step 6: Launch Full Production Training Run (50 Epochs)**
   Launch background training with persistent logging:
   `nohup python train.py --config configs/baseline_v1.yaml > train_baseline_v1.log 2>&1 &`
   Config: Resolution 1024×1024, Batch Size 4, Grad Accum 4 (Effective 16), `bfloat16`, 4 workers, SceneBalancedSampler.
7. **Step 7: Checkpoint Evaluation & Model Export (Post-Training)**
   Shubham evaluates `checkpoints/best_model.pt` and exports the frozen model to Track 2 ONNX:
   `python evaluate.py --checkpoint checkpoints/best_model.pt --device cuda`
   `python deploy/export_onnx.py --checkpoint checkpoints/best_model.pt`
8. **Step 8: Submission Packaging & Official Leaderboard Upload**
   Arin packages Track 1 predictions, runs resolution restoration, and generates the validated ZIP archive:
   `python tools/make_submission_track1.py --output submission_track1_v1.zip`
